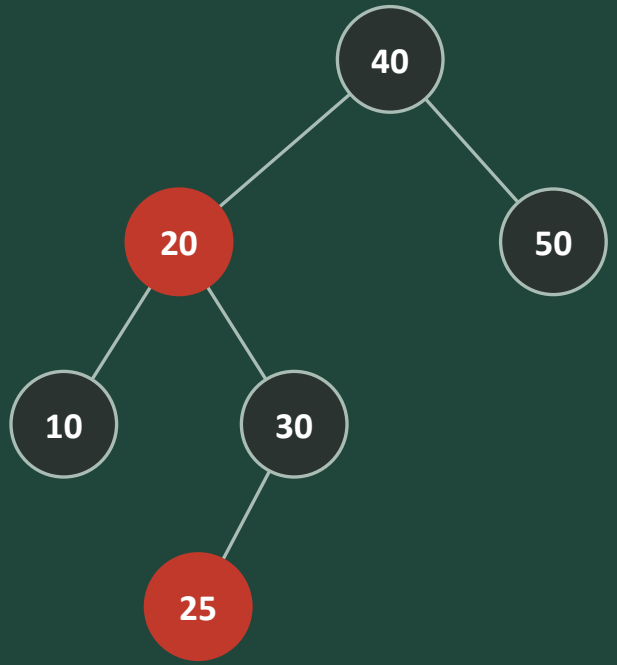
Árvores AVL e Rubro-Negra para rastreio de encomendas da Black Friday



Trabalho da Unidade II - EDB 2

Mateus Dias Oliveira - 20250034167

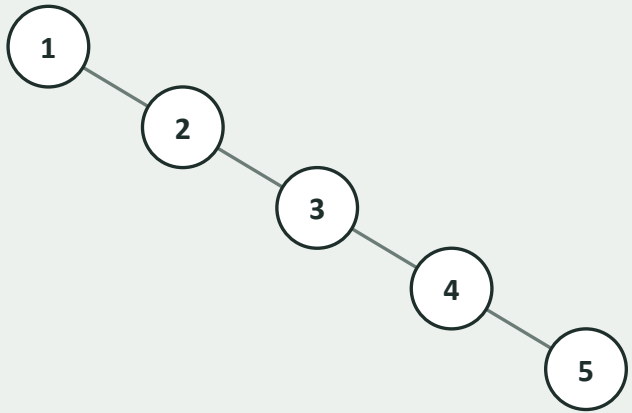
A proposta

Na Black Friday, o sistema de logística recebe um volume muito grande de identificadores de encomendas. Cada um precisa ser registrado, localizado e, depois da entrega ou do cancelamento, removido.

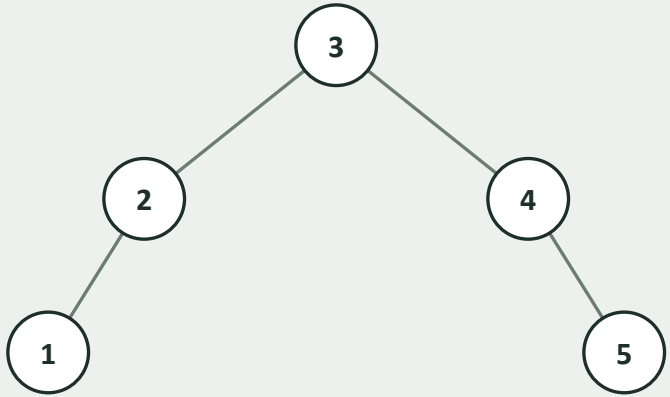
O problema é que a ordem de chegada não depende da gente. Identificadores gerados em sequência chegam em ordem crescente, justamente o caso que mais prejudica uma árvore de busca comum.

O desafio técnico: por que balancear

Sem balanceamento · altura 5



Com balanceamento · altura 3



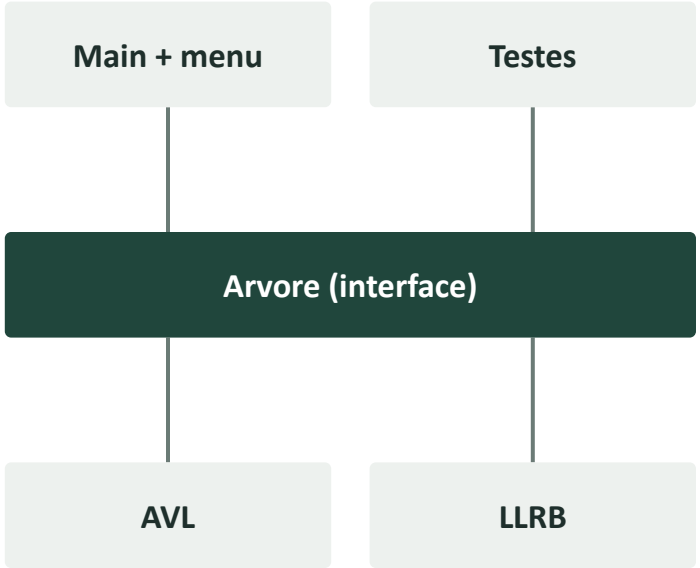
Sem balanceamento, buscar, inserir e remover custam O(n). Balanceada, a árvore garante O(log n) em qualquer ordem de chegada

n	Pior caso sem balancear	Altura mínima
1.000	1.000 comparações	10
100.000	100.000 comparações	17

A solução desenvolvida

Opção	Função
1	Inserir (valor repetido é ignorado e avisado)
2	Remover (valor inexistente é avisado)
3	Buscar
4	Percurso em ordem
5	Visualizar a estrutura (na rubro-negra mostra as cores)
6	Estatísticas: altura, rotações, inversões de cor e validação

Nenhum método lança exceção em uso normal. inserir, remover e buscar devolvem boolean, e o menu recusa o que não for número inteiro



Árvore AVL: como balanceia

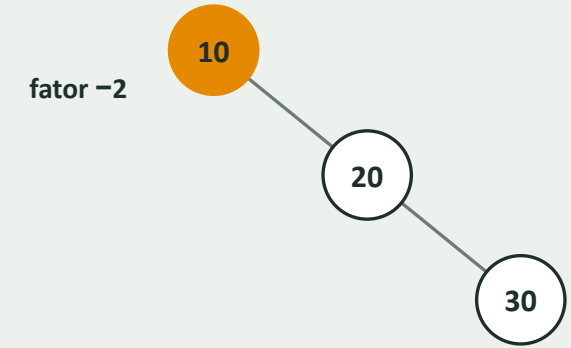
Em todo nó, a diferença entre as alturas das subárvores esquerda e direita é de no máximo 1. Cada nó guarda a própria altura. Depois de inserir ou remover, os nós do caminho recalculam a altura e rotacionam caso o fator saia do intervalo

```
fator = altura(esq) − altura(dir)
```

Caso	Correção
Esquerda-esquerda	Rotação simples à direita
Direita-direita	Rotação simples à esquerda
Esquerda-direita	Rotação à esquerda no filho, depois à direita
Direita-esquerda	Rotação à direita no filho, depois à esquerda

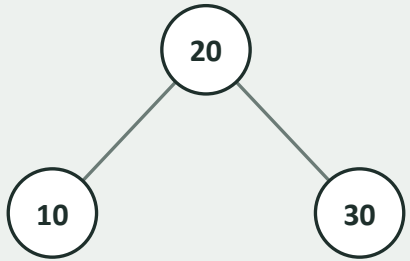
AVL: exemplo de rotação

Antes: ao inserir o 30



rotação simples à esquerda

Depois da rotação




```
      30
20
      10
```

Saída real do programa (opção 5). A árvore sai deitada, ou seja, a subárvore direita fica em cima e a raiz na coluna 0.

Estatísticas: altura 2 e 1 rotação.

Rubro-negra caída para a esquerda

Variante da rubro-negra em que todo link vermelho pende para a esquerda, equivalente a uma árvore 2-3 desenhada como binária. Um nó novo entra vermelho e os ajustes ao lado restauram as propriedades.

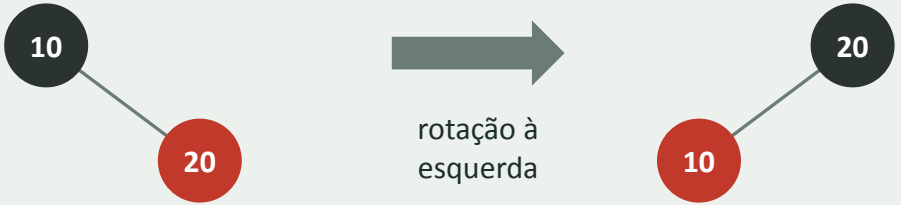
1. Todo nó é vermelho ou preto.
2. A raiz é preta.
3. Nenhum filho direito é vermelho.
4. Nenhum nó vermelho tem filho esquerdo vermelho.
5. Todo caminho da raiz até uma folha tem o mesmo número de nós pretos.

Quando	Faz
Filho direito vermelho e esquerdo preto	Rotação à esquerda
Filho esquerdo vermelho com filho esquerdo vermelho	Rotação à direita
Os dois filhos vermelhos	Inversão de cores: o vermelho sobe um nível

LLRB: ajuste de cores e rotação

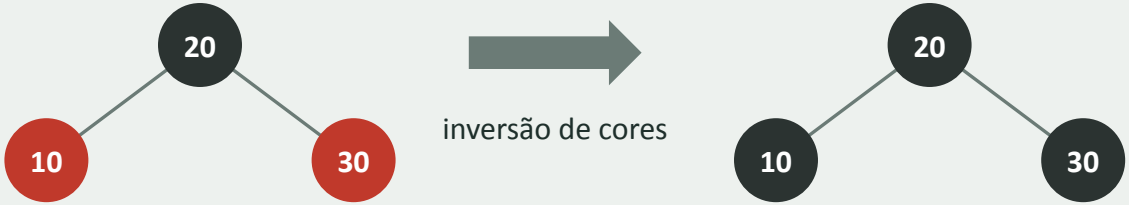
Inserir 10 e 20

O 20 entra à direita e vermelho, o que quebra a regra do filho direito. Correção: rotação à esquerda



Inserir 30

Os dois filhos do 20 ficam vermelhos. Correção: inversão de cores, e a raiz continua preta



Remoção nas duas árvores

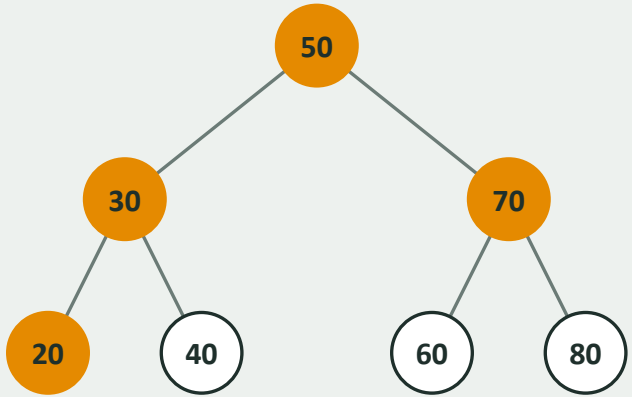
AVL

Remove como numa árvore de busca comum. Com dois filhos, a chave é trocada pela do sucessor em-ordem, que sai no lugar. Na volta, cada nó recalcula a altura e rotaciona se precisar

Rubro-negra (LLRB)

Empurra um nó vermelho para baixo durante a descida, para o nó removido nunca ser um preto isolado. Depois, balancear arruma o caminho na volta

Árvore usada nos testes



Removidos nesta ordem: 20 (folha), 30 (um filho), 70 (dois filhos) e 50 (raiz).

Também testados: valor inexistente, árvore vazia e árvore com um elemento.

Demonstração do sistema

```
AVL
10 inserido.
20 inserido.
30 inserido.
20 já existe; inserção ignorada.
99 não encontrado.
99 não existe; nada foi removido.
10 removido.
20 30
Altura: 2
Rotações: 1
Inversões de cor: 0
Propriedades preservadas: sim
```

```
Rubro-negra · antes de remover o 40
    50 (P)
40 (P)
        30 (P)
            25 (V)
    20 (V)
        10 (P)

depois de remover o 40
    50 (P)
30 (P)
        25 (P)
    20 (V)
        10 (P)

Altura: 3 · Rotações: 6
Inversões de cor: 4
```

Testes realizados

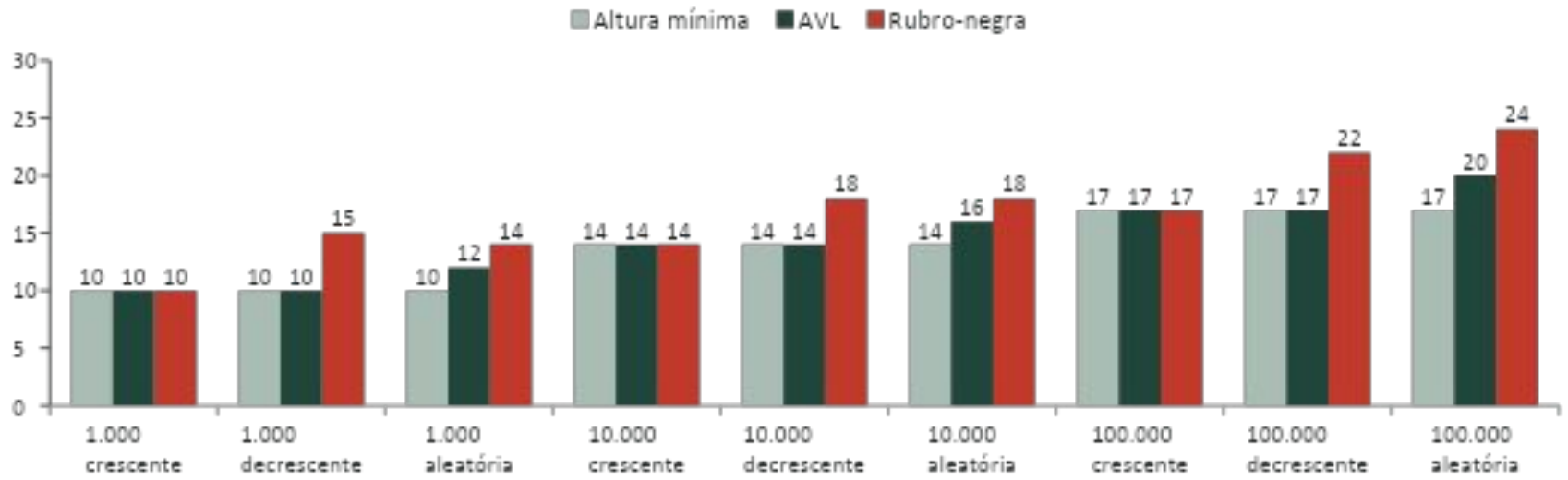
Cenário	Resultado
Árvore vazia: buscar, remover e visualizar	false, false e "(arvore vazia)"
Um único elemento: inserir, repetir e remover	ok
Busca de existente e de inexistente	ok
Valor repetido	rejeitado, árvore intacta
Remoção de folha, um filho, dois filhos e raiz	ok, propriedades preservadas
Remoção de inexistente	false, árvore intacta
1.000 inserções crescentes, decrescentes e aleatórias	válida após cada inserção
20.000 operações aleatórias contra o TreeSet	iguais, válida a cada operação

O que valida() confere

AVL: ordem de busca, fator entre −1 e 1 e altura guardada correta.

LLRB: ordem, raiz preta, sem filho direito vermelho, sem dois vermelhos seguidos e mesma altura negra.

Comparação: altura das árvores



A AVL fica sempre perto do mínimo e nunca passou da rubro-negra. Na rubro-negra o pior caso é a inserção decrescente: 22 níveis contra 17 com n = 100.000. Os limites teóricos foram respeitados; com n = 100.000 aleatório, AVL 20 (limite ≈ 24) e rubro-negra 24 (limite ≈ 33).

Comparação: rotações e ajustes

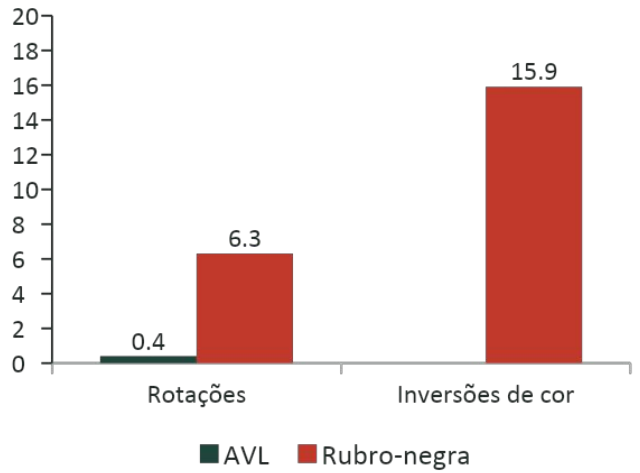
Totais com n = 100.000

Cenário	Rot. AVL	Rot. RN	Inv. cor RN
Crescente	99.983	99.984	99.978
Decrescente	99.983	99.978	99.978
Aleatória	69.920	118.191	74.440
Aleatória + remoção de metade	90.970	433.509	869.400

Com inserções ordenadas o custo é quase o mesmo, perto de uma rotação por inserção

Com inserções aleatórias, a rubro-negra faz 1,7x mais rotações. Na remoção a diferença é grande, como mostra o gráfico

Por remoção (50.000 remoções)



A diferença vem do algoritmo de remoção da LLRB. Uma rubro-negra clássica faz até 3 rotações por remoção. Contei operações, não medi tempo.

AVL e rubro-negra lado a lado

	AVL	Rubro-negra (LLRB)
Critério de balanceamento	Altura: diferença de no máximo 1	Cores: mesma altura negra em todo caminho
Altura máxima	≈ 1,44 · $\log_2 n$	≈ 2 · $\log_2 n$
Altura medida (n = 100.000)	17 a 20	17 a 24
Custo da inserção	0 a 2 rotações	Rotações e inversões de cor
Custo da remoção (medido)	≈ 0,4 rotação	≈ 6,3 rotações e 15,9 inversões
Memória por nó	A altura (um inteiro)	A cor (1 bit)
Implementação	Mais simples de entender	Remoção mais difícil
Indicada para	Muitas buscas e poucas alterações	Muitas inserções e remoções

Recomendação: AVL

Busca mais rápida

A altura da AVL foi menor ou igual em todos os testes, com até 5 níveis a menos que a LLRB na entrada decrescente. O rastreio consulta muito mais do que altera.

Remoção mais barata

Cerca de 0,4 rotação por remoção contra 6,3 na LLRB, em 50.000 remoções.

Altura previsível

A altura da AVL quase não depende da ordem de chegada. A da LLRB foi de 17 a 22 conforme a ordem.

Repositório

Repositório

https://github.com/mateusnoites/ArvoresEDB

Código-fonte, versão compilada (pasta bin) e README com instruções.

```
javac -d bin *.java
java -cp bin Main
java -cp bin Testes
```

Mateus Dias Oliveira
20250034167 - EDB 2